workleap

How we failed at Event Sourcing

Louis Beaudoin-Allaire

9.3

We did Event Sourcing.

9.3

In Production.
For 8 years.

9.3

Because it made sense.

9.3

We wouldn't do it again.

9.3

Not because it's bad.

9.3

Because we paid a cost we didn't need.

Today's story

Why Event Sourcing seemed promising

Where it hurt us

The mistakes we made

What we would do differently

Hi! I'm Louis!

- Principal Software Developer at Workleap
- Owner of tech strategy for ShareGate
- Passionate about
  - Software Architecture
  - Simple UX
  - Teamwork

https://www.linkedin.com/in/louis-beaudoin-allaire/

Definitions

Event what?

Event Driven vs Event Sourced

Event Driven

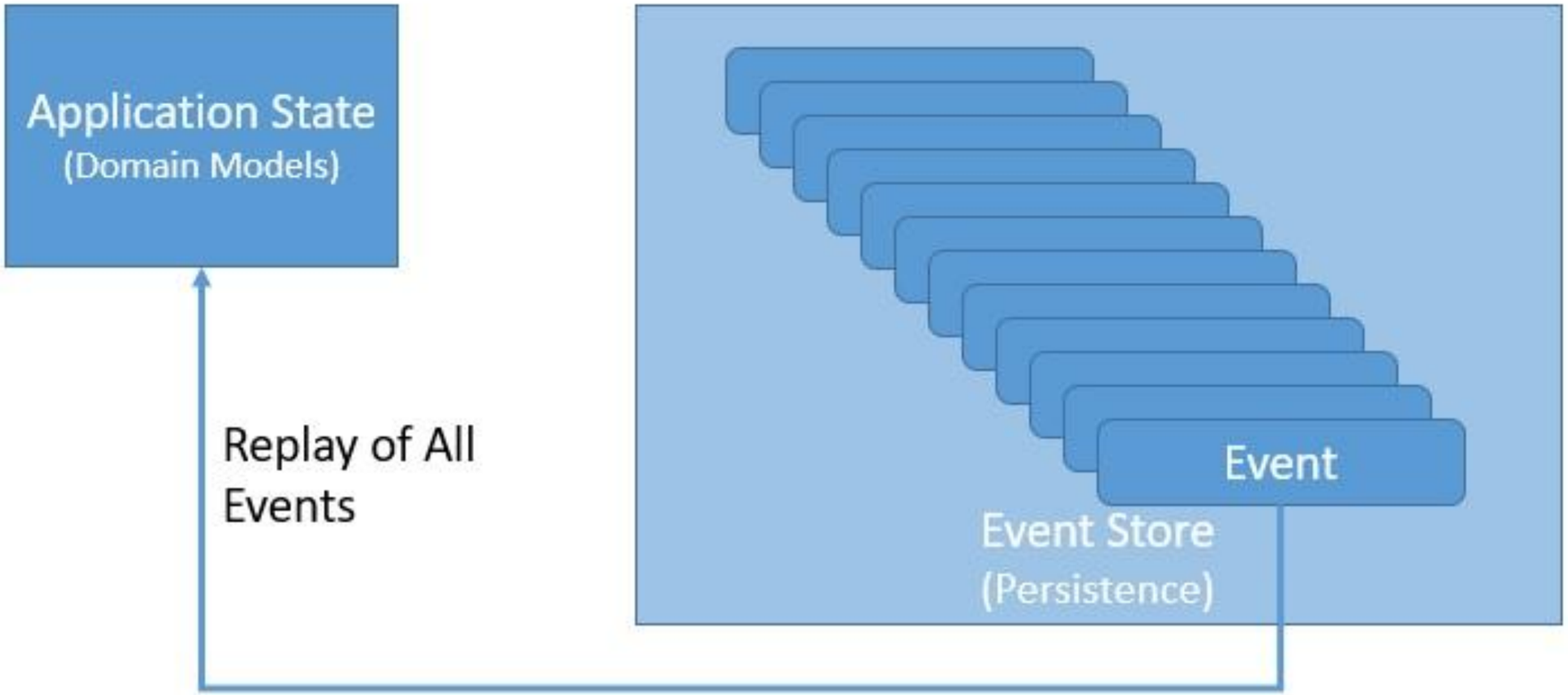
Notifies other modules/services via events or messages.

Events are short-lived.

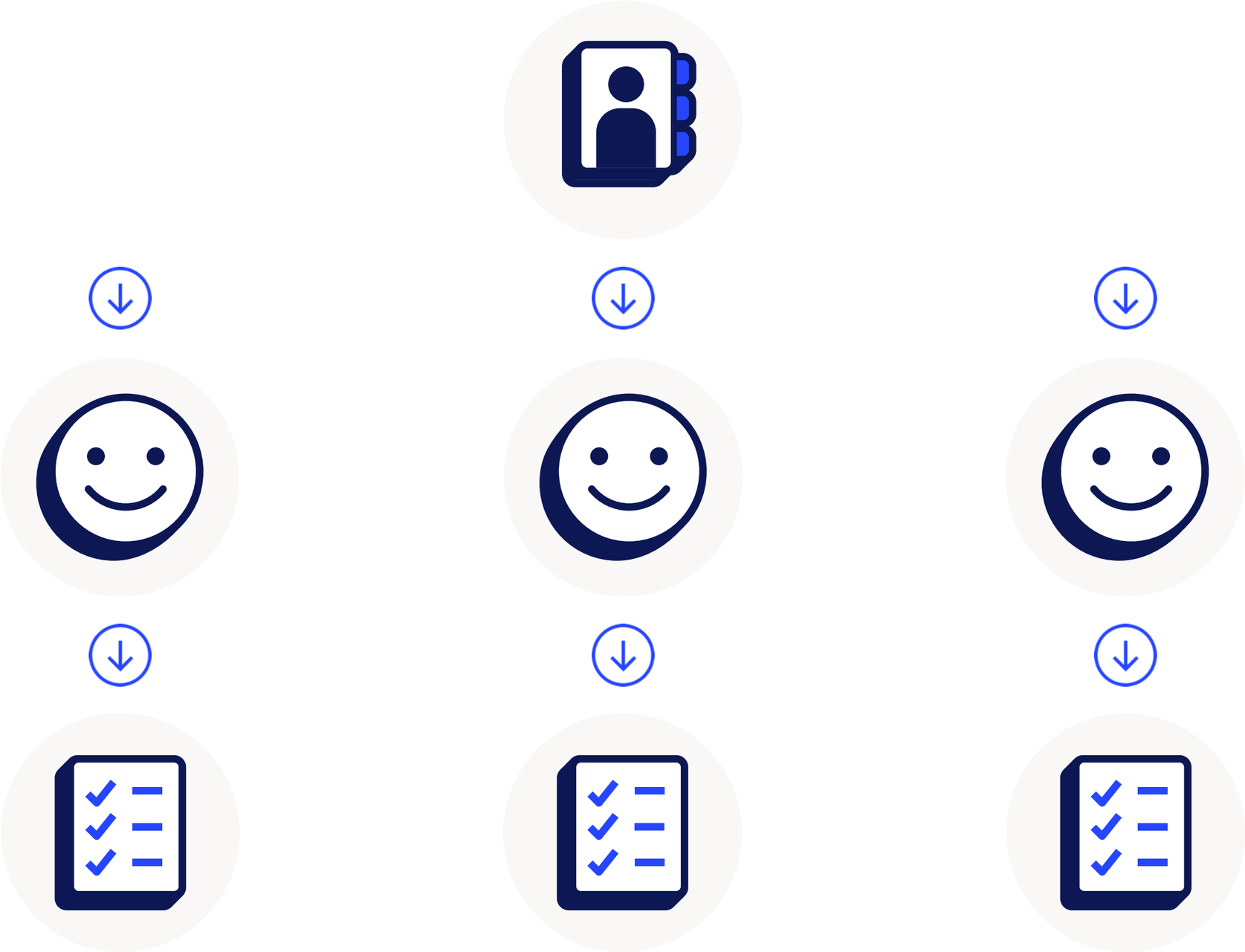
Event Sourced

Notifies other modules/services via events.

Events are the source of truth of the system.
Events are a persisted and immutable.
State is the replayed history of events.



https://danylomeister.wordpress.com/2020/06/25/cqs-cqrs-event-sourcing-whats-the-difference/



Event Sourcing is attractive.

- Audit trail
- Rebuildable state
- Time travel
- Reduces coupling

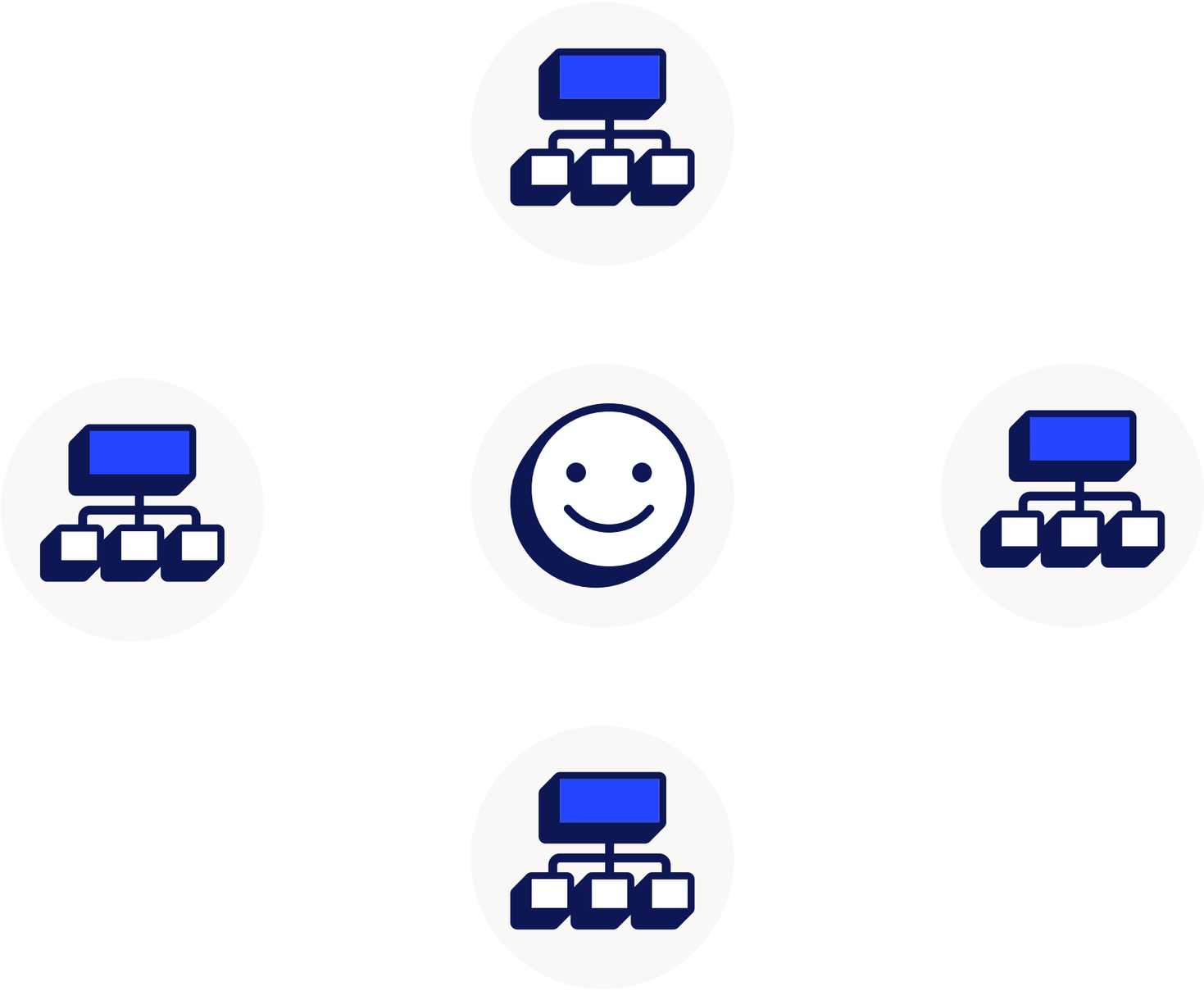
What went wrong?

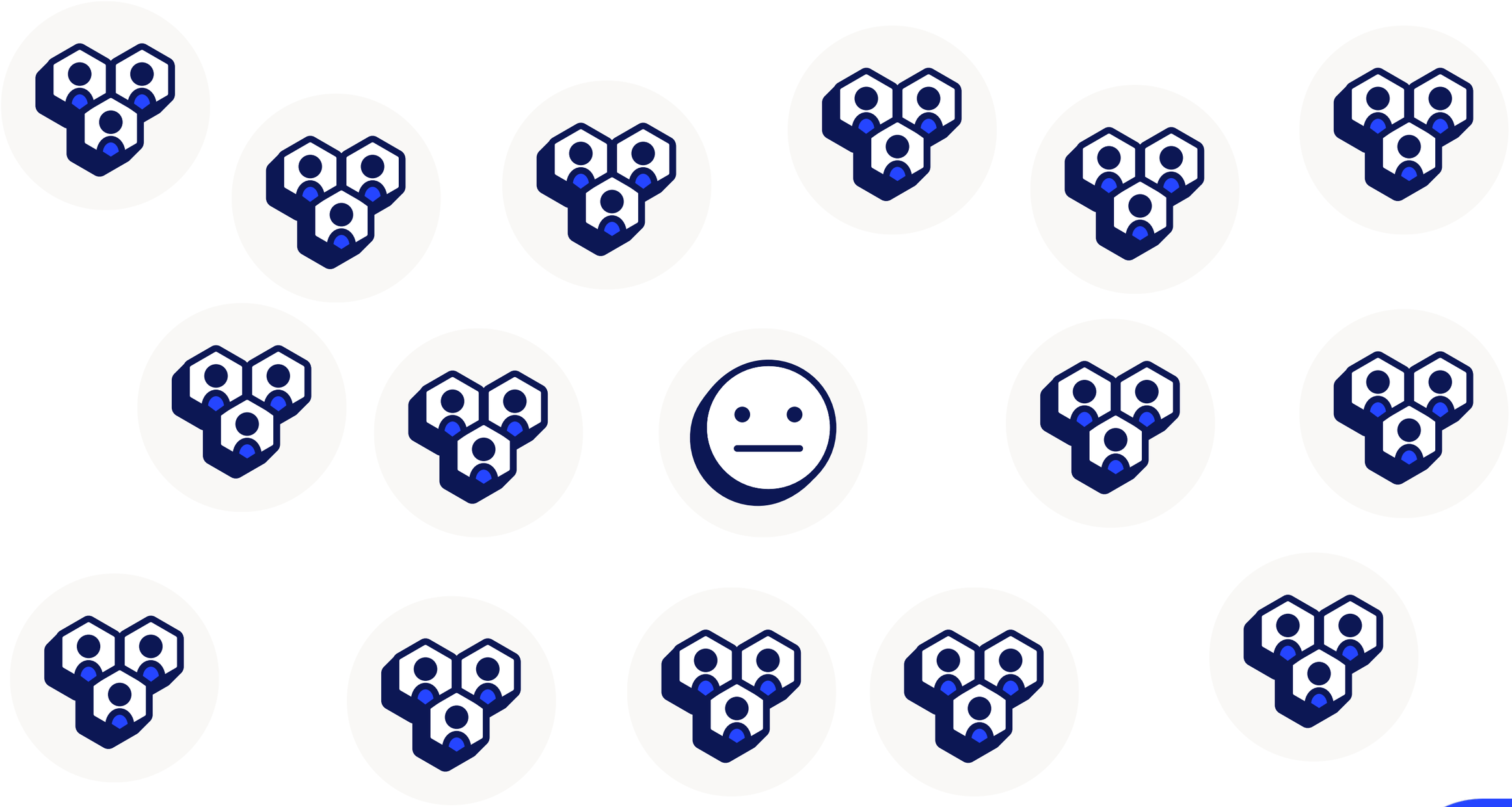
It seems perfect, doesn’t it?

Scenario: Orgs moving to M365

- Hierarchical
- IT-driven
- On-Premise

- Flat hierarchy
- Self-serve
- Cloud





We built a great feature, but...

Helps IT govern Microsoft 365 Groups.

- Doesn't exactly fit in product vision
- Hybrid infrastructure
- Potential for an entire product

We need to build.

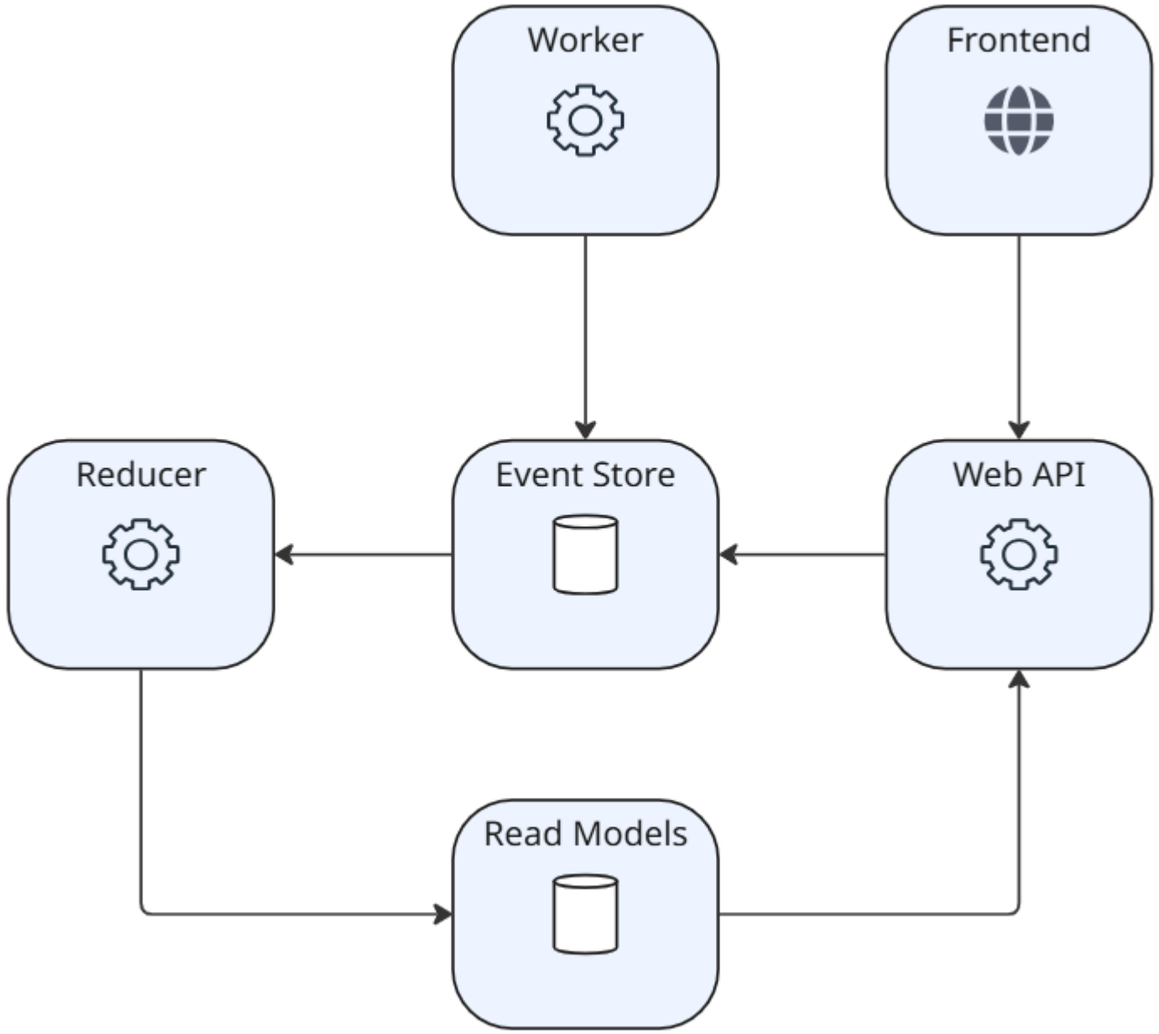
And we need it fast.

Backend architect

Frontend architect

Security specialist

Infra/DevOps specialist


Worker
Frontend
Reducer
Event Store
Web API
Read Models

At first, it worked.

- Architecture felt clean
- Good boundaries
- Low coupling
- High confidence
- Applying the "recipe" works

Growing is not scaling.

We need more!

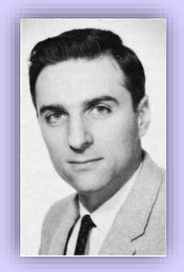
More features!

We need more!

More teams!

We need more!

“ Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure. ”



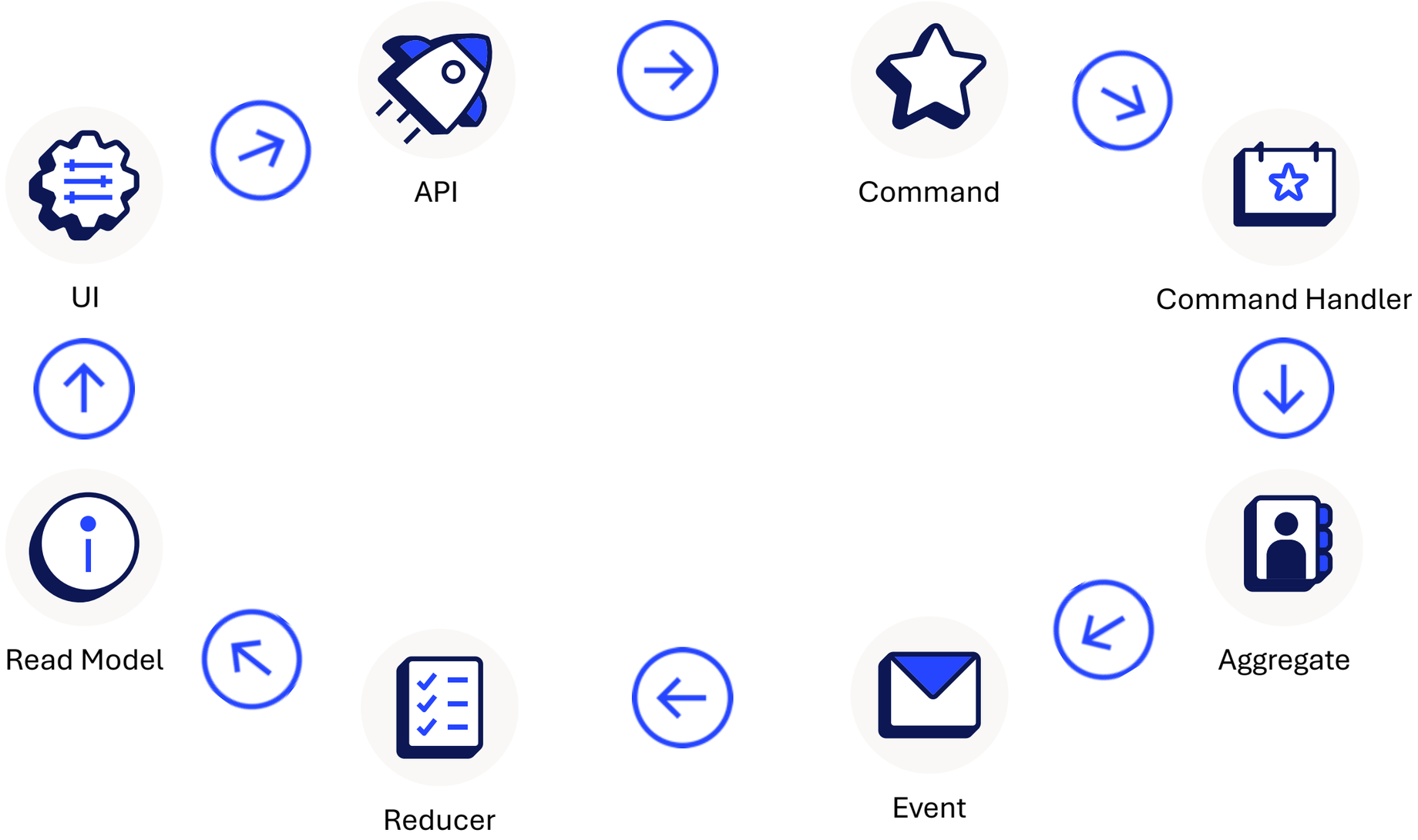
Melvin Conway
1968

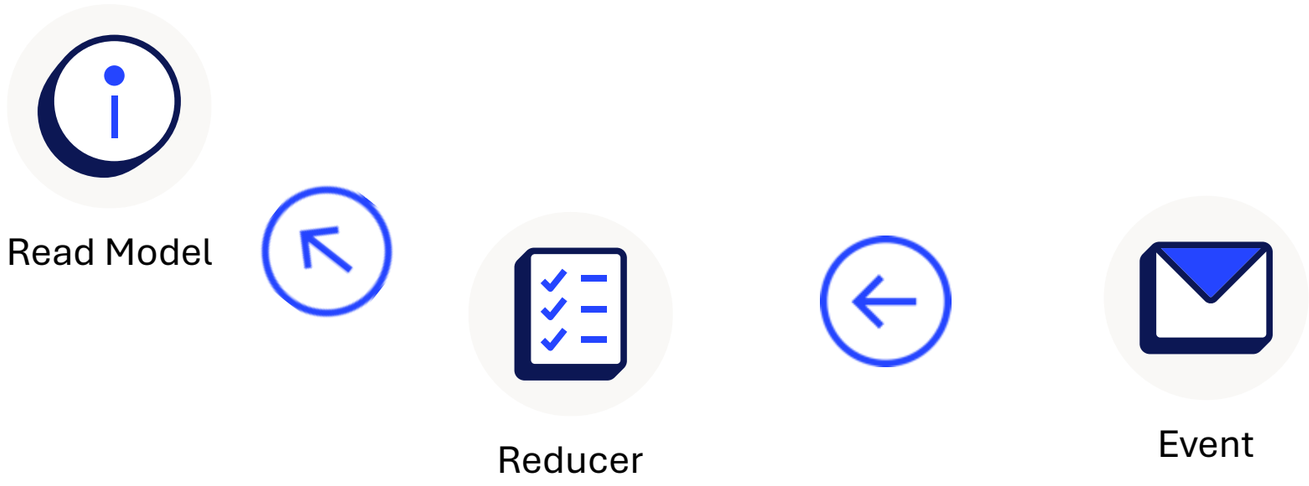
Conway's law in effect

- Work assigned based on user journey
- Alignment on tech
- Lack of alignment on domains
- Leads to
  - → Duplicated work
  - → Less cohesive boundaries

“ Why is the recipe so complicated? ”

Senior Software Developer


UI
API
Command
Command Handler
Aggregate
Event
Reducer
Read Model


Read Model
Reducer
Event

9.3

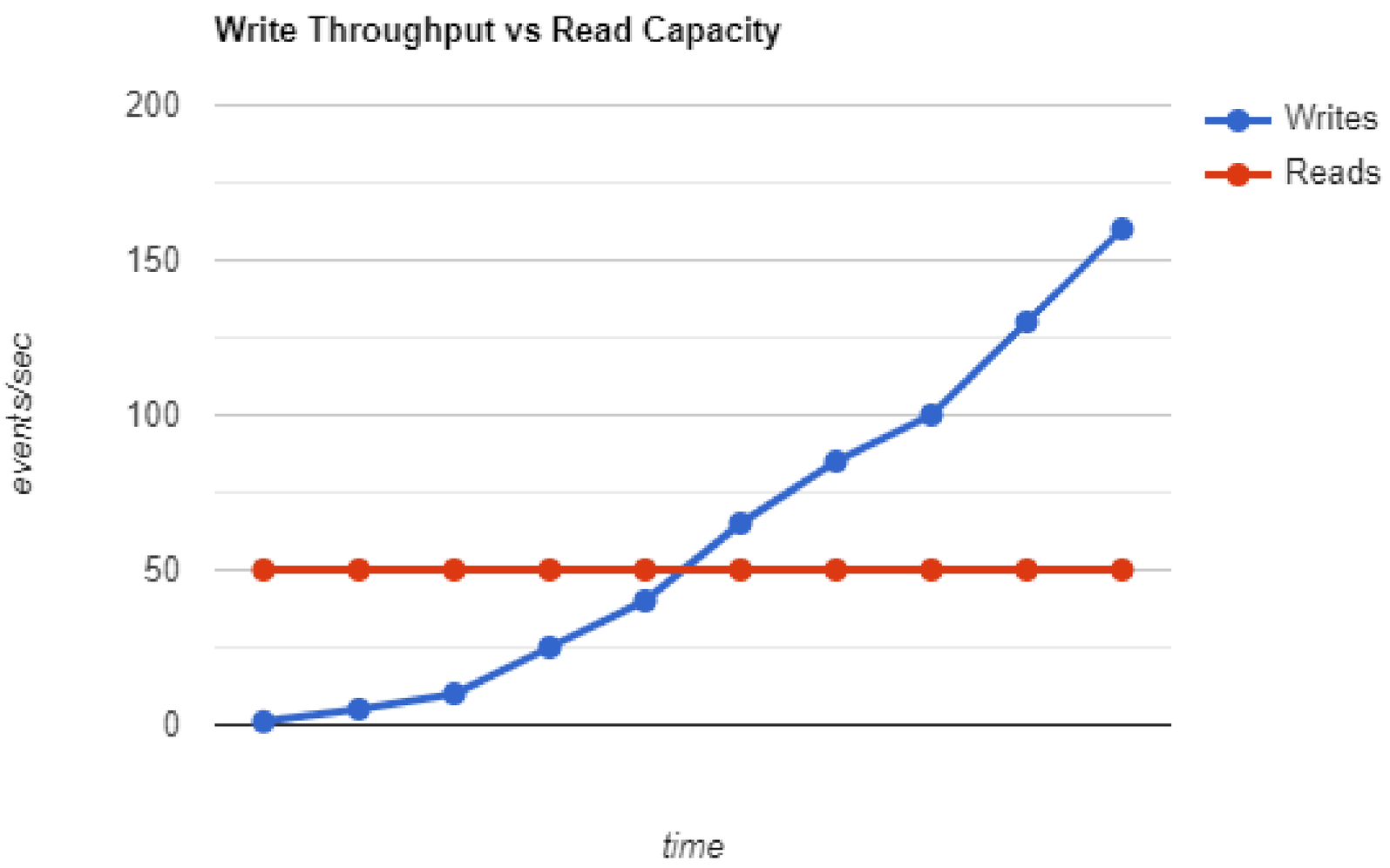
Eventual consistency.

The *hidden* assumption

Events are cheap and we can always replay the history.

Everything became an event

- Detecting a new Group?
- Detecting Owners/Members?
- Detecting activity?


Write Throughput vs Read Capacity
200
150
100
50
0
events/sec
time
Writes
Reads

At risk when...

Large tenants onboard

Burst of events generated.

Events total growing

Observed growth of 1.5x each 6 month.

Lack of isolation

Noisy neighbors

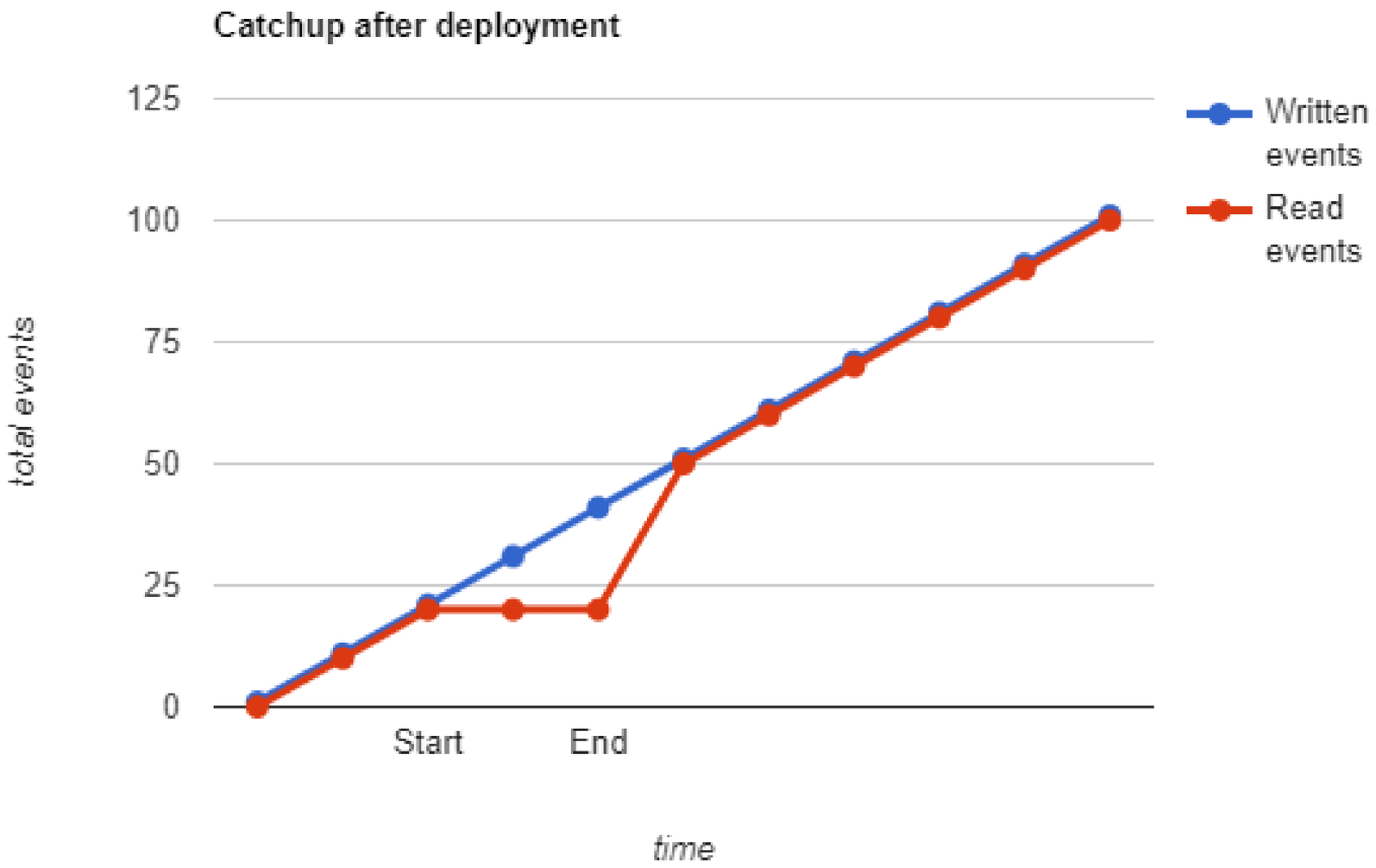
Onboarded tenants could encounter performance issues caused by larger tenants.

Saturated connections

Event Store DB cannot handle enough distinct connections for all customers.

Let's catchup

Because you can't always be live.


Catchup after deployment
125
100
75
50
25
0
total events
Start
End
time
Written events
Read events

After you deployed

1. Find where you left off.

Determine what your last processed event is. Whether you acted or not.

Ex: Save the event number that generated the read model's update.

2. Read missed events

Processed all events until you reach the head of the stream.

3. Go live.

Process events as they are saved.

It's taking a while...

Could take up to 15 minutes at times.

- Causes a "readonly" experience
- Cannot leverage full CI/CD
- Rebuilding from the start now takes weeks

Let's try to fix this...

- Snapshots
- Sharding
- Stateful actor model
- Reducing event quantity

The fixes have costs

Added complexity

Complex solutions to complex problems.

Knowledge silos

Feature teams want to focus on user-facing value.

New needs

Fresh new ideas.

- Assumption it will be part of the monolith
- Not directly related to the domain.

“ How about trying something different? ”

Me

So, we did.

Simpler architecture

Use of a leaner CRUD architecture on an independent infrastructure.

Happier developers

More flexibility, allowed to review the technical stack.

Faster delivery

Increase in velocity, ability to pivot and no longer felt held back.

9.3

The architecture became the product.

Constraint to accommodate

Support more types

No longer just Groups. We want other workspaces like Teams, Sites, OneDrive, etc.

Parity of features

Every feature we have so far, we want to support for all.

Changes to existing behaviors

Need flexibility and ability to rapidly bring changes.

We planned a refactor.

- Rewrite modules gradually
- Move traffic to new services
- Backwards compatibility became an issue.

A new day

Let's rethink what we're building.

Let's start over

Retire the product

We announced the decommission to our customers.

Management was retired August 2025.

Launch Protect

We took the opportunity to think about today's reality and the needs of our customers.

Protect officially launched November 2025.

9.3

1 610 814 205

9.3

This isn't an anti-Event Sourcing talk.

Event Sourcing

Is risky

Domain isn't clear and will evolve.
Unpredictable event volume.

Demands

Discipline, order, vision and governance.

What we would do differently.

- Better (earlier) observability.
- Think longer term.
- Persist necessity, not curiosity.

9.3

Event Sourcing didn't fail us.

Thank you!

workleap